

Question 1

Given the recurrence relation:

$$T(n) = T(n/2) + T(2n/5) + 7n$$

With base case $T(0) = 1$.

- **Analysis:** The fractions inside the arguments add up to $1/2 + 2/5 = 9/10$. Because this sum is strictly less than 1, the total work is entirely dominated by the linear overhead term ($7n$).

Result: $\Theta(n)$

Question 2

For constants $a \geq 1$ and $b > 1$:

$$T(n) = aT(n/b) + f(n)$$

Under what conditions will $T(n) = \Theta(n \log n)$ hold?

- **Analysis:** By the Master Theorem, to get a tight bound containing a single logarithm multiplier ($\log n$), the driving function must match the core dividing factor ($n^{(\log_b a)}$). Equating the exponents forces $a = b$, which leaves the overhead function to grow linearly.

Result: $a = b$ and $f(n) = \Theta(n)$

Question 3

An algorithm checks whether an array of size N is sorted by making a single pass and comparing only adjacent elements. What is the tightest running time?

- **Analysis:** A single pass checking every neighbor sequentially takes exactly $N - 1$ comparison operations.

Result: $\Theta(N)$

Question 4

Evaluate the asymptotic order of:

$$1^3 + 2^3 + 3^3 + \dots + n^3$$

- **Analysis:** The mathematical formula for the sum of cubes is $[n(n+1)/2]^2$, which expands to a polynomial where the largest exponent is n^4 .
- **Correct Options:** $\Theta(n^4)$, $O(n^5)$, and $\Omega(n^3)$

Question 5

Find the final value of q:

```
int fun1(int n)
{
    int i, j, k, p, q=0;
    for(i=1; i<n; ++i)
    {
        p=0;
        for(j=n; j>1; j=j/2)
            ++p;
        for(k=1; k<p; k=k*2)
            ++q;
    }
    return q;
}
```

- **Analysis:** The outer loop runs n times. The first inner loop continuously halves n , making $p = \log_2 n$. The second inner loop doubles k until it reaches p , running $\log_2 p$ times per iteration. Substituting p gives $\log_2(\log_2 n)$ increments per cycle.

Result: $\Theta(n \log \log n)$

Question 6

Determine the complexity:

```
for(int i=n; i>0; i/=2)
{
    for(int j=0; j<i; j++)
    {
        // constant work
    }
}
```

- **Analysis:** The loop boundaries shrink geometrically: $n + n/2 + n/4 + n/8 + \dots$. This infinite series converges to a total value of $2n$.

Result: $\Theta(n)$

Question 7

Determine the complexity:

```

for(int i=1;i≤n;i++)
{
    for(int j=1;j≤i;j++)
    {
        // constant work
    }
}

```

- **Analysis:** This represents a simple triangular loop combination totaling $1 + 2 + 3 + \dots + n$, which evaluates directly to $n(n+1)/2$.

Result: $\Theta(n^2)$

Question 8

Determine the complexity:

```

void fun(int n)
{
    if(n≤1)
        return;
    fun(n/2);
    fun(n/2);
}

```

- **Analysis:** This maps to the recurrence structure $T(n) = 2T(n/2) + \Theta(1)$. According to the Master Theorem, the total leaf-node operations dominate the constant overhead.

Result: $\Theta(n)$

Question 9

Determine the complexity:

```

int fib(int n)
{
    if(n≤1)
        return n;
    return fib(n-1)+fib(n-2);
}

```

- **Analysis:** This creates a cascading tree of binary recursive calls that grows exponentially. It tracks the golden ratio growth rate (1.618^n), or can be broadly upper-bounded by binary doubling.

Result: $\Theta(1.618^n)$ or $O(2^n)$

Question 10

Arrange the following in increasing order:

$n^{3/2}$, $n \log n$, $n^{\log_2 n}$, 2^n

Result: $n \log n < n^{3/2} < n^{\log_2 n} < 2^n$

Question 11

Determine the complexity:

$$T(n) = T(n-1) + 1$$

Result: $\Theta(n)$

Question 12

Determine the complexity:

$$T(n) = T(n-1) + n$$

Result: $\Theta(n^2)$

Question 13

Determine the complexity:

$$T(n) = 2T(n-1) + 1$$

Result: $\Theta(2^n)$

Question 14

Determine the complexity:

$$T(n) = T(n/2) + 1$$

Result: $\Theta(\log n)$

Question 15

Determine the complexity:

```
for(int i=1;i<=n;i*=2)
{
    for(int j=1;j<=n;j++)
    {
```

```
}  
}
```

- **Analysis:** The outer loop doubles its counter every time, executing a total of $\log_2 n$ times. The inner loop runs a steady n times during every single iteration.

Result: $\Theta(n \log n)$